SIMATS ENGINEERING
Approved by AICTE | IET-UK Accreditation

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Operating System Management

Process Scheduling • Memory Allocation • I/O Operations

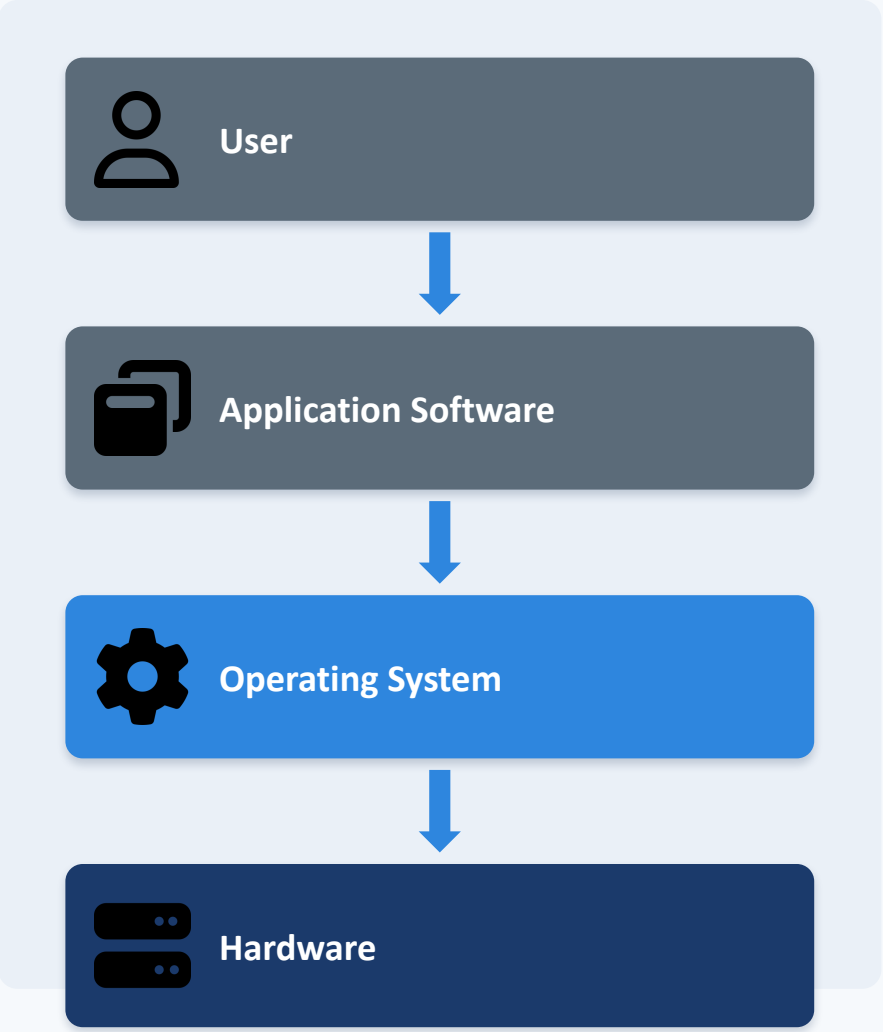
PRESENTED BY **Saimeera D**

COURSE **CSA0401-OPERATING SYSTEM**

GUIDED BY **DR. J.Ruby Elizabeth**

Introduction to Operating System

- Definition: System software that manages computer hardware and software resources
- Purpose: Acts as a bridge between the user and the computer hardware
- Types: Batch, Time-Sharing, Distributed, Real-Time, and Mobile OS
- Importance: Enables multitasking, security, and efficient resource sharing
- Without an OS, every application would need to control hardware directly



Key Functions of an Operating System

Process Management

Creates, schedules, and terminates processes

Memory Management

Allocates and tracks RAM for running programs

File System Management

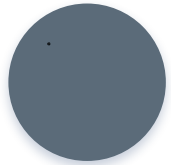
Organizes, stores, and retrieves files on disk

I/O Device Management

Controls communication with input/output devices

Security & Protection

Guards data and resources from unauthorized access



Resource Allocation

Distributes CPU, memory, and devices fairly

Process Scheduling

- Process: a program in execution, with its own memory and state
- CPU Scheduling: OS decides which process runs next on the CPU
- Context Switching: saving and restoring process state on a switch
- Scheduling Goals: fairness, high CPU use, low waiting time

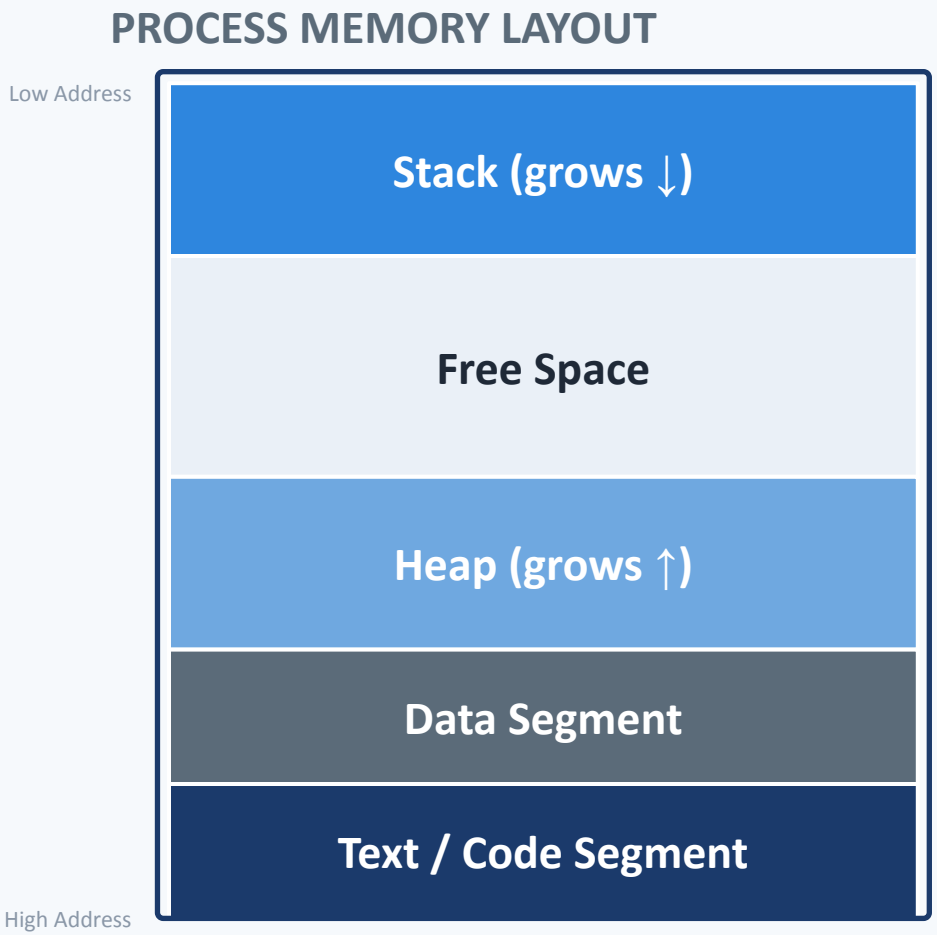
COMMON SCHEDULING ALGORITHMS

FCFS | SJF | Priority | Round Robin

PROCESS STATE DIAGRAM

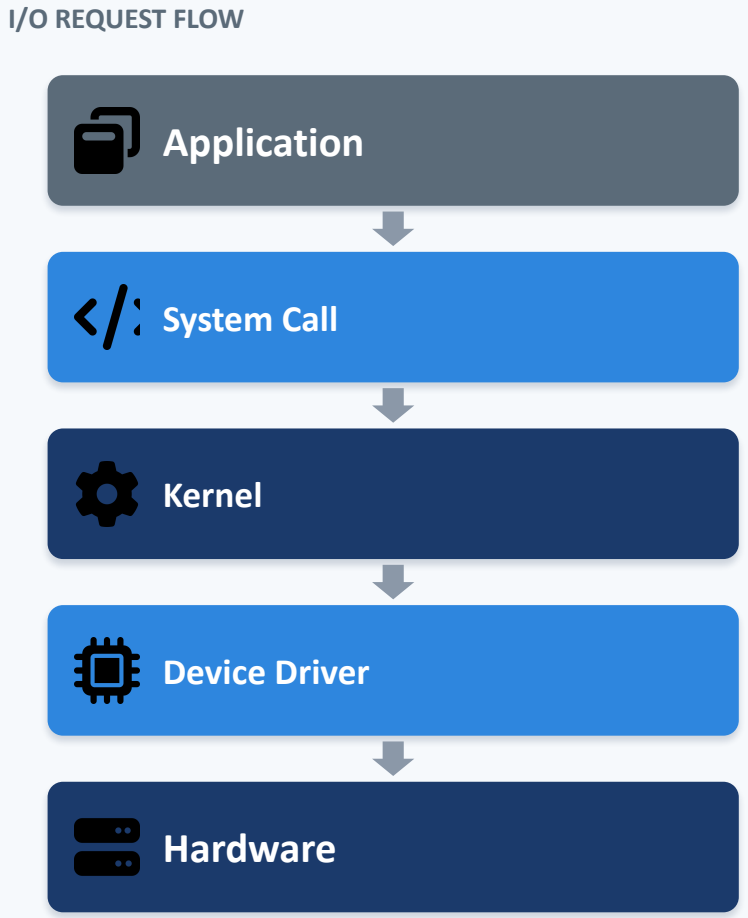
Memory Allocation

- RAM: fast, temporary storage that holds running programs and data
- Virtual Memory: uses disk space to extend available RAM
- Paging: divides memory into small fixed-size blocks called pages
- Segmentation: divides a program into logical parts (code, data, stack)
- Swapping: moves processes between RAM and disk as needed
- Memory Protection: stops one process from accessing another's memory



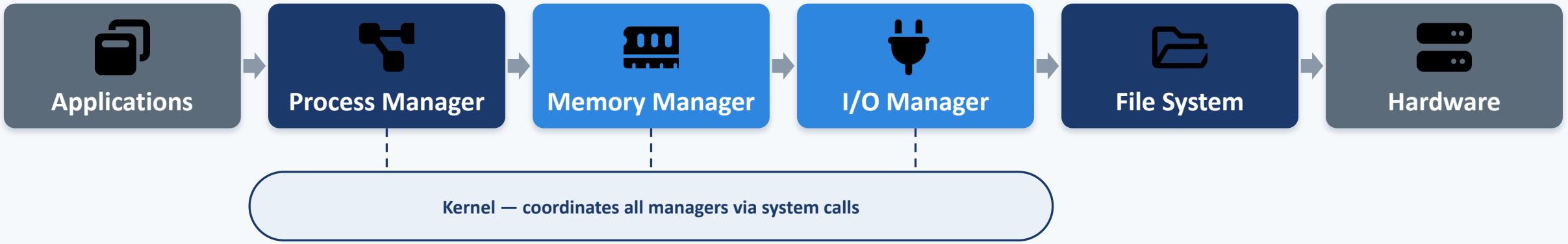
I/O Operations

- Input Devices: keyboard, mouse, scanner send data into the system
- Output Devices: monitor, printer, speaker deliver results to the user
- Device Drivers: software that lets the OS talk to specific hardware
- Interrupts: signals that pause the CPU to handle urgent device events
- DMA: lets devices transfer data directly to memory, bypassing the CPU
- Buffering & Spooling: temporarily hold data to smooth out I/O speed



Interaction Between OS Components

All managers communicate through the kernel using system calls and shared data structures



- Applications request services (e.g., open a file) through system calls
- The kernel routes each request to the correct manager (process, memory, I/O, or file system)
- Managers coordinate with each other and directly control the hardware

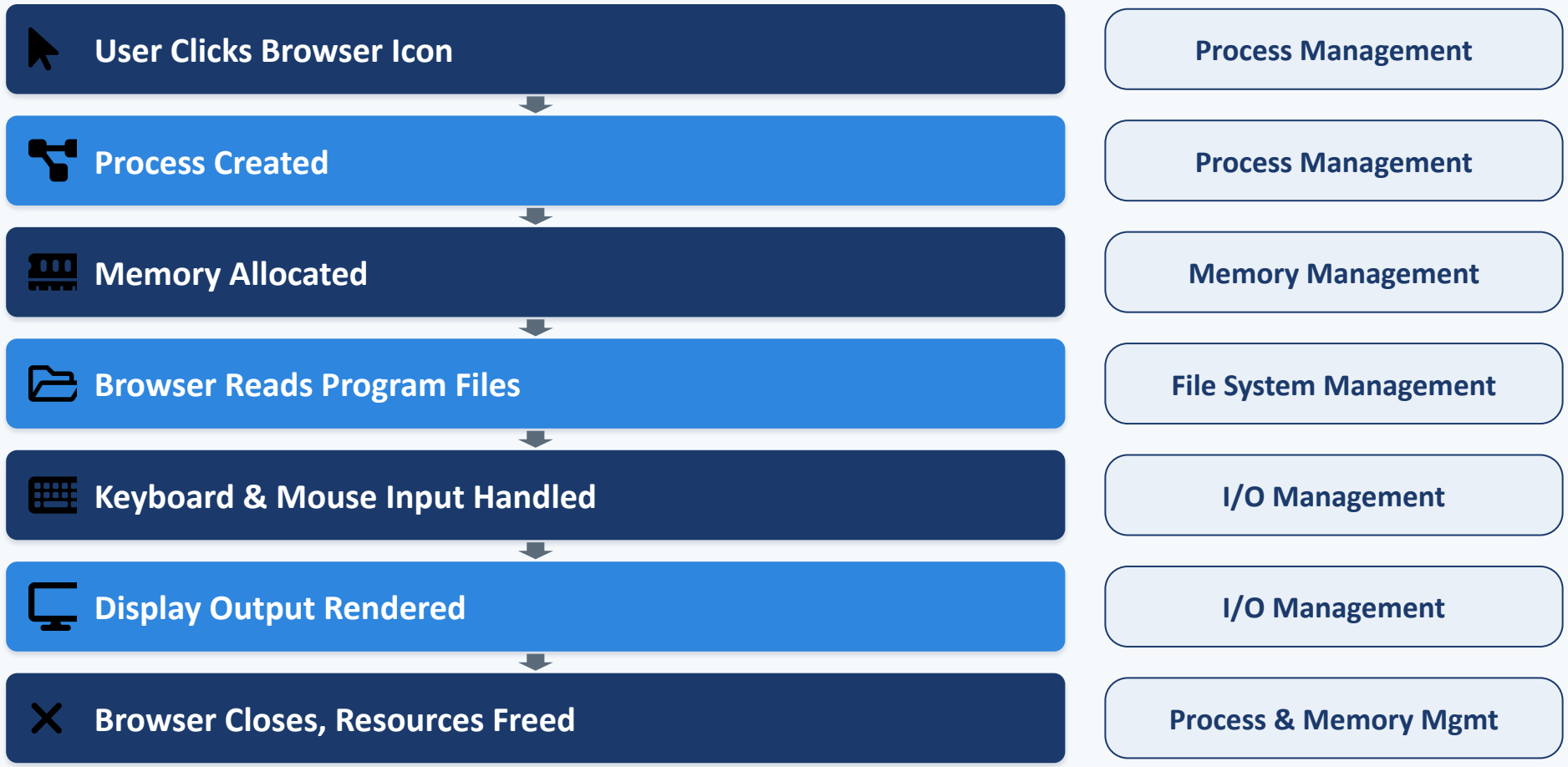
Examples of System Calls

OS Function	System Calls	Purpose
Process Management	`fork(), exec(), wait(), exit()`	Create and manage processes
Memory Management	`malloc(), free(), mmap(), brk()`	Allocate and release memory
File Management	`open(), read(), write(), close()`	Perform file operations
I/O Management	`ioctl(), read(), write()`	Handle device communication

PRACTICAL EXAMPLE

- fork() — a shell creates a new process when you open a terminal tab
- malloc() — a program requests memory to store an array or object
- open()/read() — a text editor opens and reads a document's contents
- ioctl() — the OS configures a webcam or printer during use

Real-World Example: Opening a Web Browser



Conclusion

- Process Scheduling ensures fair and efficient CPU usage
- Memory Allocation keeps programs isolated and running smoothly
- I/O Operations connect the system to users and devices
- OS Components interact constantly through the kernel
- System Calls are the bridge between programs and the OS